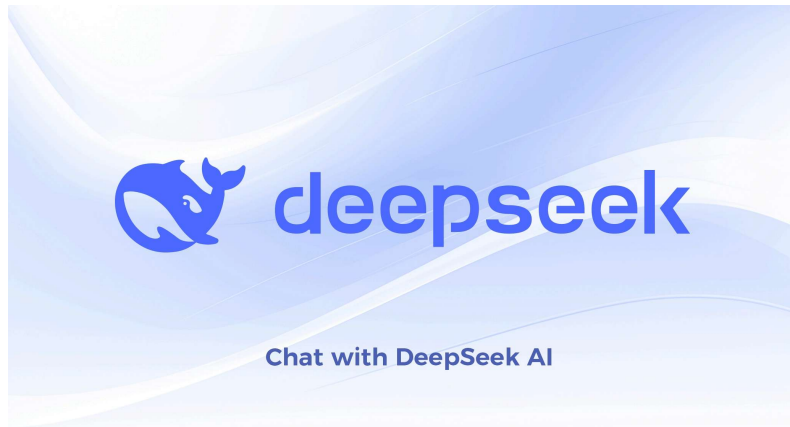


Express JS Interview Concepts Guide



Express.js Complete Interview Questions with Solutions

Here's a comprehensive collection of Express.js interview questions categorized by difficulty level, with detailed solutions and code examples.

BASIC LEVEL QUESTIONS

Q1: What is Express.js and why do we use it?

Answer: Express.js is a minimal and flexible Node.js web application framework that provides robust features for building web and mobile applications. It simplifies the development of server-side applications by providing:

- Easy routing mechanisms
- Middleware support
- Template engine integration
- HTTP utility methods
- RESTful API development [-1-8](#)

javascript

```
const express = require('express');
const app = express();

app.get('/', (req, res) => res.send('Hello World'));
app.listen(3000);
```

Q2: What are the key features of Express.js?

Answer:

- **Lightweight and fast** - Minimal overhead
- **Middleware support** - Functions that execute during request-response cycle
- **Routing** - Define endpoints for different HTTP methods
- **Template engines** - Support for EJS, Pug, Handlebars
- **Static file serving** - Serve CSS, images, JavaScript files
- **Error handling** - Built-in and custom error handling
- **Database integration** - Works with MongoDB, MySQL, PostgreSQL [-1-9](#)

Q3: How do you create a basic Express server?

Answer:

javascript

```
const express = require('express');
const app = express();
const PORT = process.env.PORT || 3000;

app.get('/', (req, res) => {
  res.send('Welcome to Express Server');
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Q4: What is middleware in Express.js?

Answer: Middleware functions are functions that have access to the request object (req), response object (res), and the next middleware function in the application's request-response cycle [-3-7](#).

javascript

```
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url} - ${new Date()}`);
  next();
});

const authMiddleware = (req, res, next) => {
  if (req.headers.authorization) {
    next();
  } else {
    res.status(401).json({ error: 'Unauthorized' });
  }
};
```

Q5: What is the difference between `app.get()` and `app.use()`?

Answer:

javascript

```
app.use('/api', (req, res, next) => {
  console.log('API middleware');
  next();
});

app.get('/api/users', (req, res) => {
  res.json({ users: [] });
});
```



INTERMEDIATE LEVEL QUESTIONS

Q6: How do you handle file uploads in Express?

Answer: Using `multer` middleware for handling multipart/form-data [-2-4](#).

javascript

```
const multer = require('multer');
const path = require('path');

const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, 'uploads/');
  },
  filename: (req, file, cb) => {
    cb(null, Date.now() +
    path.extname(file.originalname));
  }
});

const fileFilter = (req, file, cb) => {
  const allowedTypes = ['image/jpeg', 'image/png',
  'image/jpg'];
  if (allowedTypes.includes(file.mimetype)) {
    cb(null, true);
  } else {
    cb(new Error('Invalid file type'), false);
  }
};

const upload = multer({
  storage: storage,
  limits: { fileSize: 5 * 1024 * 1024 },
  fileFilter: fileFilter
});

app.post('/upload', upload.single('avatar'), (req, res) => {
  res.json({ file: req.file });
});

app.post('/uploads', upload.array('photos', 5), (req, res) => {
  res.json({ files: req.files });
});
```

Q7: How do you implement JWT authentication in Express?

Answer: Using `jsonwebtoken` package for token-based authentication [-2-5](#).

javascript

```
const jwt = require('jsonwebtoken');
const bcrypt = require('bcryptjs');

const SECRET_KEY = process.env.JWT_SECRET || 'your-secret-key';

app.post('/register', async (req, res) => {
  const { email, password } = req.body;

  const hashedPassword = await bcrypt.hash(password, 10);

  const user = { id: Date.now(), email, password:
hashedPassword };

  res.status(201).json({ message: 'User created
successfully' });
});

app.post('/login', async (req, res) => {
  const { email, password } = req.body;

  const user = { id: 1, email: 'user@example.com' };

  const token = jwt.sign(
    { userId: user.id, email: user.email },
    SECRET_KEY,
    { expiresIn: '1h' }
  );

  res.json({ token, expiresIn: 3600 });
});

const authenticateToken = (req, res, next) => {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) {
    return res.status(401).json({ error: 'Access token
required' });
  }

  jwt.verify(token, SECRET_KEY, (err, user) => {
    if (err) {
```

```
        return res.status(403).json({ error: 'Invalid or
expired token' });
    }
    req.user = user;
    next();
  });
};

app.get('/profile', authenticateToken, (req, res) => {
  res.json({ user: req.user });
});
```

Q8: How do you enable CORS in Express?

Answer: Using the `cors` middleware package [-3-8](#).

javascript

```
const cors = require('cors');

app.use(cors());

const corsOptions = {
  origin: ['https://example.com',
    'https://api.example.com'],
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization'],
  credentials: true,
  optionsSuccessStatus: 200
};

app.use(cors(corsOptions));

app.get('/public-data', cors(), (req, res) => {
  res.json({ data: 'Public data' });
});

const dynamicCors = (req, callback) => {
  const allowedOrigins = ['https://allowed1.com',
    'https://allowed2.com'];
  const origin = req.header('Origin');

  if (allowedOrigins.includes(origin)) {
    callback(null, { origin: true });
  } else {
    callback(null, { origin: false });
  }
};

app.use(cors(dynamicCors));
```

Q9: How do you handle errors globally in Express?

Answer: Using error-handling middleware with 4 parameters [-2-7](#).

javascript

```
app.use((err, req, res, next) => {
  console.error(err.stack);

  const status = err.status || 500;
  const message = err.message || 'Internal Server Error';

  res.status(status).json({
    success: false,
    error: {
      message: message,
      status: status,
      ...(process.env.NODE_ENV === 'development' && {
        stack: err.stack })
    }
  });
});

class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
    this.isOperational = true;

    Error.captureStackTrace(this, this.constructor);
  }
}

const asyncHandler = (fn) => (req, res, next) => {
  Promise.resolve(fn(req, res, next)).catch(next);
};

app.get('/user/:id', asyncHandler(async (req, res) => {
  const user = await findUser(req.params.id);
  if (!user) {
    throw new AppError('User not found', 404);
  }
  res.json(user);
}));

app.use((req, res) => {
  res.status(404).json({ error: 'Route not found' });
});
```

Q10: How do you use Express Router for modular routes?

Answer: `express.Router()` creates modular, mountable route handlers [-1-3](#).

javascript

```
const express = require('express');
const router = express.Router();

router.use((req, res, next) => {
  console.log('User routes accessed');
  next();
});

router.get('/', (req, res) => {
  res.json({ users: [] });
});

router.get('/:id', (req, res) => {
  res.json({ userId: req.params.id });
});

router.post('/', (req, res) => {
  res.status(201).json({ message: 'User created' });
});

router.put('/:id', (req, res) => {
  res.json({ message: `User ${req.params.id} updated` });
});

router.delete('/:id', (req, res) => {
  res.json({ message: `User ${req.params.id} deleted` });
});

module.exports = router;

const userRoutes = require('./routes/userRoutes');
const productRoutes = require('./routes/productRoutes');

app.use('/api/users', userRoutes);
app.use('/api/products', productRoutes);
```

ADVANCED LEVEL QUESTIONS

Q11: How do you implement rate limiting in Express?

Answer: Using `express-rate-limit` middleware to prevent abuse [4](#).

javascript

```
const rateLimit = require('express-rate-limit');
const RedisStore = require('rate-limit-redis');
const Redis = require('ioredis');

const redisClient = new Redis({
  host: 'localhost',
  port: 6379
});

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 100,
  message: 'Too many requests from this IP',
  standardHeaders: true,
  legacyHeaders: false
});

app.use('/api/', limiter);

const advancedLimiter = rateLimit({
  store: new RedisStore({
    sendCommand: (...args) => redisClient.call(...args),
  }),
  windowMs: 60 * 1000,
  max: 10,
  keyGenerator: (req) => {
    return req.user?.id || req.ip;
  },
  skip: (req) => {
    return req.path === '/health';
  },
  handler: (req, res) => {
    res.status(429).json({
      error: 'Rate limit exceeded',
      retryAfter: Math.ceil(req.rateLimit.resetTime /
1000)
    });
  }
});

app.post('/login', rateLimit({
  windowMs: 60 * 60 * 1000,
  max: 5,
  message: 'Too many login attempts'
}), loginHandler);
```

Q12: How do you optimize Express application performance?

Answer: Multiple strategies for performance optimization [-4-7](#).

javascript

```
const compression = require('compression');
const helmet = require('helmet');
const cluster = require('cluster');
const os = require('os');

app.use(compression({
  level: 6,
  threshold: 1024,
  filter: (req, res) => {
    if (req.headers['x-no-compression']) {
      return false;
    }
    return compression.filter(req, res);
  }
}));

app.use(helmet());

const NodeCache = require('node-cache');
const cache = new NodeCache({ stdTTL: 600 });

const cacheMiddleware = (duration) => (req, res, next) => {
  const key = req.originalUrl;
  const cachedResponse = cache.get(key);

  if (cachedResponse) {
    return res.json(cachedResponse);
  }

  res.originalJson = res.json;
  res.json = (body) => {
    cache.set(key, body, duration);
    res.originalJson(body);
  };
  next();
};

app.get('/api/data', cacheMiddleware(300), (req, res) => {
  res.json({ data: 'Cached response' });
});

if (cluster.isMaster) {
  const numCPUs = os.cpus().length;
  console.log(`Master ${process.pid} setting up ${numCPUs}
workers`);
}
```

```
for (let i = 0; i < numCPUs; i++) {
  cluster.fork();
}

cluster.on('exit', (worker) => {
  console.log(`Worker ${worker.process.pid} died,
restarting`);
  cluster.fork();
});
} else {
  app.listen(3000, () => {
    console.log(`Worker ${process.pid} started`);
  });
}

const mysql = require('mysql2');
const pool = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: 'password',
  database: 'app_db',
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});

app.use((req, res, next) => {
  res.setHeader('Cache-Control', 'public, max-age=3600');
  next();
});
```

Q13: How do you implement WebSockets with Express?

Answer: Using [Socket.IO](#) integration with Express [-5](#).

javascript

```
const express = require('express');
const http = require('http');
const socketIo = require('socket.io');
const jwt = require('jsonwebtoken');

const app = express();
const server = http.createServer(app);
const io = socketIo(server, {
  cors: {
    origin: "http://localhost:3000",
    methods: ["GET", "POST"]
  }
});

io.use((socket, next) => {
  const token = socket.handshake.auth.token;

  try {
    const decoded = jwt.verify(token,
process.env.JWT_SECRET);
    socket.user = decoded;
    next();
  } catch (err) {
    next(new Error('Authentication error'));
  }
});

io.on('connection', (socket) => {
  console.log(`User ${socket.user.id} connected`);

  socket.join(`user_${socket.user.id}`);

  socket.on('message', (data) => {

    io.emit('message', {
      userId: socket.user.id,
      message: data.message,
      timestamp: new Date()
    });
  });

  socket.on('private-message', ({ to, message }) => {
    io.to(`user_${to}`).emit('private-message', {
      from: socket.user.id,
      message: message
    });
  });
});
```

```
    });  
  });  
  
  socket.on('typing', ({ isTyping }) => {  
    socket.broadcast.emit('user-typing', {  
      userId: socket.user.id,  
      isTyping  
    });  
  });  
  
  socket.on('disconnect', () => {  
    console.log(`User ${socket.user.id} disconnected`);  
    io.emit('user-offline', { userId: socket.user.id });  
  });  
  
  app.post('/api/notify/:userId', authenticateToken, (req, res)  
=> {  
    const { userId } = req.params;  
    const { notification } = req.body;  
  
    io.to(`user_${userId}`).emit('notification',  
notification);  
    res.json({ success: true });  
  });  
  
  server.listen(3000);
```

Q14: How do you handle database transactions in Express?

Answer: Using Mongoose for MongoDB or connection pools for SQL databases [-4](#).

javascript

```
const mongoose = require('mongoose');

const transferMoney = async (fromAccount, toAccount, amount)
=> {
  const session = await mongoose.startSession();
  session.startTransaction();

  try {

    await Account.findOneAndUpdate(
      { accountNumber: fromAccount },
      { $inc: { balance: -amount } },
      { session }
    );

    await Account.findOneAndUpdate(
      { accountNumber: toAccount },
      { $inc: { balance: amount } },
      { session }
    );

    await Transaction.create([
      {
        from: fromAccount,
        to: toAccount,
        amount: amount,
        timestamp: new Date()
      }
    ], { session });

    await session.commitTransaction();
    return { success: true };
  } catch (error) {
    await session.abortTransaction();
    throw error;
  } finally {
    session.endSession();
  }
};

app.post('/api/transfer', authenticateToken, async (req, res,
next) => {
  try {
    const result = await transferMoney(
      req.body.fromAccount,
      req.body.toAccount,
      req.body.amount
    );
    res.json(result);
  }
}
```

```
    } catch (error) {
      next(error);
    }
  });

const { Pool } = require('pg');
const pool = new Pool();

app.post('/api/order', authenticateToken, async (req, res) => {
  const client = await pool.connect();

  try {
    await client.query('BEGIN');

    const orderResult = await client.query(
      'INSERT INTO orders (user_id, total) VALUES ($1, $2) RETURNING id',
      [req.user.id, req.body.total]
    );

    const orderId = orderResult.rows[0].id;

    for (const item of req.body.items) {
      await client.query(
        'INSERT INTO order_items (order_id, product_id, quantity) VALUES ($1, $2, $3)',
        [orderId, item.productId, item.quantity]
      );

      await client.query(
        'UPDATE products SET stock = stock - $1 WHERE id = $2',
        [item.quantity, item.productId]
      );
    }

    await client.query('COMMIT');
    res.json({ orderId, success: true });
  } catch (error) {
    await client.query('ROLLBACK');
    throw error;
  } finally {
    client.release();
  }
});
```

Q15: How do you implement logging and monitoring in Express?

Answer: Using Morgan for HTTP logging and Winston for application logging [-4-7.](#)

javascript

```
const morgan = require('morgan');
const winston = require('winston');
const DailyRotateFile = require('winston-daily-rotate-file');

const logger = winston.createLogger({
  level: 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.json(),
    winston.format.errors({ stack: true })
  ),
  transports: [
    new winston.transports.File({ filename:
'logs/error.log', level: 'error' }),
    new winston.transports.File({ filename:
'logs/combined.log' }),
    new DailyRotateFile({
      filename: 'logs/application-%DATE%.log',
      datePattern: 'YYYY-MM-DD',
      maxSize: '20m',
      maxFiles: '14d'
    })
  ]
});

if (process.env.NODE_ENV !== 'production') {
  logger.add(new winston.transports.Console({
    format: winston.format.simple()
  }));
}

const morganFormat = morgan('combined', {
  stream: {
    write: (message) => {
      logger.info(message.trim());
    }
  }
});

app.use(morganFormat);

app.use((req, res, next) => {
  const start = Date.now();

  logger.info({
```

```
        type: 'request',
        method: req.method,
        url: req.url,
        ip: req.ip,
        userAgent: req.get('User-Agent')
    });

    res.on('finish', () => {
        const duration = Date.now() - start;
        logger.info({
            type: 'response',
            method: req.method,
            url: req.url,
            status: res.statusCode,
            duration: `${duration}ms`
        });
    });
});

next();
});

app.get('/health', (req, res) => {
    const health = {
        status: 'healthy',
        timestamp: new Date(),
        uptime: process.uptime(),
        memory: process.memoryUsage(),
        cpu: process.cpuUsage()
    };

    res.json(health);
});

const responseTime = require('response-time');
app.use(responseTime((req, res, time) => {
    if (time > 1000) {
        logger.warn({
            type: 'slow_request',
            method: req.method,
            url: req.url,
            duration: `${time}ms`
        });
    }
})));
```

PRACTICAL CODING SCENARIOS

Scenario 1: Build a REST API with CRUD Operations

javascript

```
const express = require('express');
const router = express.Router();
let items = [];
let nextId = 1;

router.post('/items', (req, res) => {
  const item = { id: nextId++, ...req.body, createdAt: new
Date() };
  items.push(item);
  res.status(201).json(item);
});

router.get('/items', (req, res) => {
  const page = parseInt(req.query.page) || 1;
  const limit = parseInt(req.query.limit) || 10;
  const startIndex = (page - 1) * limit;
  const endIndex = page * limit;

  const paginatedItems = items.slice(startIndex, endIndex);
  res.json({
    page,
    limit,
    total: items.length,
    totalPages: Math.ceil(items.length / limit),
    data: paginatedItems
  });
});

router.get('/items/:id', (req, res) => {
  const item = items.find(i => i.id ===
parseInt(req.params.id));
  if (!item) return res.status(404).json({ error: 'Item not
found' });
  res.json(item);
});

router.put('/items/:id', (req, res) => {
  const index = items.findIndex(i => i.id ===
parseInt(req.params.id));
  if (index === -1) return res.status(404).json({ error:
'Item not found' });

  items[index] = { ...items[index], ...req.body, updatedAt:
new Date() };
  res.json(items[index]);
});
```

```
router.delete('/items/:id', (req, res) => {
  const index = items.findIndex(i => i.id ===
    parseInt(req.params.id));
  if (index === -1) return res.status(404).json({ error:
    'Item not found' });

  items.splice(index, 1);
  res.status(204).send();
});

module.exports = router;
```

Scenario 2: Implement Search and Filtering

javascript

```
router.get('/products', (req, res) => {
  let {
    search,
    category,
    minPrice,
    maxPrice,
    sortBy = 'name',
    sortOrder = 'asc',
    page = 1,
    limit = 20
  } = req.query;

  let filteredProducts = [...products];

  if (search) {
    filteredProducts = filteredProducts.filter(p =>
p.name.toLowerCase().includes(search.toLowerCase()) ||
p.description.toLowerCase().includes(search.toLowerCase())
    );
  }

  if (category) {
    filteredProducts = filteredProducts.filter(p =>
p.category === category);
  }

  if (minPrice) {
    filteredProducts = filteredProducts.filter(p =>
p.price >= parseFloat(minPrice));
  }
  if (maxPrice) {
    filteredProducts = filteredProducts.filter(p =>
p.price <= parseFloat(maxPrice));
  }

  filteredProducts.sort((a, b) => {
    let comparison = 0;
    if (a[sortBy] > b[sortBy]) comparison = 1;
    if (a[sortBy] < b[sortBy]) comparison = -1;
    return sortOrder === 'desc' ? comparison * -1 :
comparison;
  });
});
```

```
const startIndex = (page - 1) * limit;
const paginatedProducts =
  filteredProducts.slice(startIndex, startIndex + limit);

res.json({
  filters: { search, category, minPrice, maxPrice },
  pagination: {
    page: parseInt(page),
    limit: parseInt(limit),
    total: filteredProducts.length,
    pages: Math.ceil(filteredProducts.length / limit)
  },
  products: paginatedProducts
});
});
```

 QUICK REFERENCE CARD

Most Common Interview Topics [-2-3-5](#)

Topic	Key Points
Middleware	app.use(), next(), order matters, error handling (4 params)
Routing	app.METHOD(), route parameters (:id), query strings (? page=1)
Authentication	JWT, bcrypt, middleware protection, token verification
Error Handling	try/catch, next(err), global error handler, 404 handler
Performance	compression, clustering, caching, connection pooling
Security	helmet, CORS, rate limiting, input validation, sanitization
Database	Connection pooling, transactions, indexing, aggregation
File Upload	multer, storage configuration, file validation, limits

Express.js Method Comparison [-6-8](#)

Key Code Snippets for Interviews

1. Basic Server:

javascript

```
const express = require('express');  
const app = express();  
app.listen(3000);
```

2. JSON Middleware:

javascript

```
app.use(express.json());  
app.use(express.urlencoded({ extended: true }));
```

3. Error Handler:

javascript

```
app.use((err, req, res, next) => {  
  res.status(500).json({ error: err.message });  
});
```

4. Static Files:

javascript

```
app.use(express.static('public'));
```

5. Environment Variables:

javascript

```
require('dotenv').config();  
const PORT = process.env.PORT || 3000;
```

This comprehensive guide covers the most frequently asked Express.js interview questions with practical solutions. Focus on understanding middleware flow, error handling patterns, and common security practices. Practice building a complete CRUD API with authentication to solidify these concepts [-5-9](#). Good luck with your interview! 🚀